

Final Project — Data Science in Cyber

Full name : Omar Hijab

Id : 212749048

github repo :

[OmarHi07/Data_Science_in_Cyber_project](https://github.com/OmarHi07/Data_Science_in_Cyber_project)

Google colab link :

https://colab.research.google.com/drive/1_WvPLpJcRIAWlQ6k1JGWan8zzhrm6h6e?usp=sharing

Summery: page 39

1. Summary of the Source

1.1 Problem Being Addressed

The selected source is the tutorial-style GitHub project “Phishing Website Detection by Machine Learning Techniques” by Shreya Gopal Sundari [1]. The project addresses the problem of automatically identifying phishing websites using machine-learning models.

Phishing is a form of social engineering in which attackers attempt to imitate trustworthy websites or URLs in order to deceive users. A victim may believe that a malicious webpage is legitimate and provide sensitive information such as login credentials, personal details, or financial information. The objective of the selected source is to build a classification system that can distinguish between phishing websites and legitimate websites based on characteristics extracted from their URLs and webpage content.

1.2 Importance of the Problem

Phishing detection is an important cybersecurity task because phishing websites can appear similar to legitimate websites and may be difficult for users to identify manually. If a phishing website is not detected, a user may disclose sensitive information or interact with malicious content.

An automated detection model could support protective systems such as web filters, browser extensions, or security tools. Such a model should identify malicious websites accurately while avoiding unnecessary blocking of legitimate websites. Therefore, both missed phishing websites and incorrectly blocked legitimate websites are important when evaluating the quality of the model.

1.3 Proposed Solution

The selected source proposes a supervised machine-learning approach. The workflow begins by collecting phishing and legitimate URLs. The project then extracts values that describe characteristics of the URL, its domain, and the webpage content. These values are used as inputs for classification models.

The source describes 17 extracted values divided into three categories:

1. Address-bar-based features: 9 values extracted directly from the URL.
2. Domain-based features: 4 values related to the website domain.
3. HTML-and-JavaScript-based features: 4 values derived from the webpage content.

The supplied processed dataset retains the textual Domain field. Before model training, the original notebook removes this field. Therefore, the classifiers actually receive 16 numerical predictors.

1.4 Dataset

The processed dataset contains 10,000 URL observations:

- 5,000 phishing observations collected from PhishTank [2].
- 5,000 legitimate observations obtained from a University of New Brunswick dataset [3].

The processed dataset is stored in the file 5.urldata.csv. Each row represents a processed URL observation and includes the extracted values and a binary target label:

- 1: phishing website.
- 0: legitimate website.

At the row level, the supplied dataset is balanced because it contains an equal number of phishing and legitimate observations. The diversity and duplication of these observations are examined later in the critical evaluation.

1.5 Models and Methodology

The source treats phishing detection as a supervised binary-classification problem. The processed dataset is divided into training and testing sets using an 80–20 split: 8,000 observations are used for training and 2,000 observations are used for testing.

The source evaluates the following models:

- Decision Tree
- Random Forest
- Multilayer Perceptron
- XGBoost
- Autoencoder Neural Network
- Support Vector Machine

According to the original source, the XGBoost classifier achieved the highest reported performance, with an accuracy of 86.4%. The trained XGBoost model was saved for possible future use in a browser extension or graphical user interface.

This reported result serves as the initial baseline for the reproduction study. The following sections evaluate whether the methodology is reproducible, whether the selected features are appropriate, and whether the reported performance remains reliable under a more detailed evaluation.

2. Critical Evaluation

2.1 Main Claims of the Source

The selected source proposes that phishing websites can be detected using supervised machine-learning models trained on features extracted from URLs, domain information, and webpage behavior. The source compares several models and reports that XGBoost achieves the highest testing accuracy of approximately 86.4%.

The project also saves the trained XGBoost classifier for possible future use in a browser extension or graphical user interface. This suggests that the proposed pipeline is intended not only as an educational experiment but also as a possible foundation for detecting new phishing URLs.

The main claims evaluated in this reproduction study are therefore:

1. The supplied dataset and code are sufficient to reproduce the reported model-training experiment.
2. XGBoost performs better than the simpler Decision Tree and Random Forest baselines.
3. The reported accuracy provides meaningful evidence that the model can distinguish phishing websites from legitimate websites.
4. The supplied feature-extraction pipeline is sufficiently reliable for applying the trained model to new URLs.

2.2 Claims Supported by the Reproduction Study

The main reported XGBoost result was reproduced successfully under an evaluation setting comparable to the original source. The source reports a testing accuracy of approximately 86.4%, while the reproduced XGBoost model achieved approximately 86.3% testing accuracy.

A deterministic controlled version of the experiment was also created using fixed random seeds and stratification. Under this controlled random split, XGBoost achieved a testing accuracy of 86.6% and outperformed the Decision Tree and Random Forest baselines.

Model	Testing Accuracy	Precision	Recall	MCC
Decision Tree	0.810	0.973	0.638	0.660
Random Forest	0.818	0.980	0.649	0.676
XGBoost	0.866	0.928	0.795	0.741

These results support the narrow claim that XGBoost performs better than the simpler tree-based models under the original random-split methodology.

However, reproducing the reported accuracy does not automatically prove that the method will generalize reliably to previously unseen websites. The following analysis identified several limitations in the dataset, evaluation methodology, and feature-extraction implementation.

2.3 Limitations of the Evaluation Methodology

2.3.1 Train-Test Contamination Caused by Duplicated Observations

The processed dataset contains 10,000 rows, but 5,626 rows are exact duplicates of earlier observations. Only 4,374 unique complete rows remain after exact duplicates are removed.

The controlled random train-test split contains substantial overlap between training and testing observations:

Overlap Check	Result
Testing rows identical to complete training rows	1,272 out of 2,000 — 63.60%
Testing rows whose domain also appears in training	1,427 out of 2,000 — 71.35%
Testing rows sharing a numerical feature vector with training	1,923 out of 2,000 — 96.15%

This means that many testing observations are identical or closely related to observations already seen during training. As a result, the random split does not provide a strong evaluation of the model's ability to generalize to genuinely unseen websites.

The issue is best described as **train-test contamination** caused by repeated observations and repeated domains. It does not prevent reproduction of the original score, but it weakens the practical meaning of that score.

2.3.2 Class Balance Depends Heavily on Duplicate Rows

The supplied dataset appears perfectly balanced because it contains 5,000 legitimate observations and 5,000 phishing observations. However, the class distribution changes substantially after exact duplicate rows are removed:

Class	Original Dataset	Deduplicated Dataset
Legitimate observations	5,000 — 50.00%	529 — 12.09%
Phishing observations	5,000 — 50.00%	3,845 — 87.91%

The legitimate class contains substantially more repeated rows than the phishing class. Therefore, the original balance reflects the number of stored rows rather than the diversity of the observations.

This distinction is important because accuracy becomes misleading when one class dominates the dataset. For example, under the deduplicated random split, the Random Forest model predicted every testing observation as phishing and still achieved an accuracy of 87.9%. However, its specificity and MCC were both equal to zero because it failed to correctly allow even one legitimate website.

Metric	Random Forest After Deduplication
Accuracy	0.879
Balanced Accuracy	0.500
Specificity	0.000
MCC	0.000

This result demonstrates that accuracy alone is insufficient for evaluating the proposed detector.

2.3.3 Generalization to Unseen Domains Is Not Demonstrated by the Original Split

To evaluate generalization more rigorously, a domain-aware split was created after removing exact duplicate rows. Each domain was assigned entirely to either the training set or the testing set. Therefore, no domain appeared in both sets.

Domain-Aware Overlap Check	Result
Exact complete-row overlap	0
Testing rows whose domain appears in training	0
Domains appearing in both sets	0

Testing rows sharing a numerical feature vector with training 792

Under this stricter evaluation, the default XGBoost model still achieved an accuracy of 87.5%. However, the confusion matrix revealed that this value was misleading:

Metric	XGBoost Under Domain-Aware Split
Accuracy	0.875
Balanced Accuracy	0.552
Precision	0.883
Recall	0.988
Specificity	0.116

Metric	XGBoost Under Domain-Aware Split
MCC	0.222
True Negatives	13
False Positives	99
False Negatives	9
True Positives	744

The model detected nearly all phishing observations, but it incorrectly blocked 99 out of 112 legitimate websites. A system with this behavior would be unsuitable for automatic blocking, even though its accuracy appears high.

A separate domain-aware validation set was used to select a more balanced threshold of 0.98. The adjusted threshold substantially improved specificity but reduced phishing recall:

Metric	Default Threshold: 0.50 Validation-Selected Threshold: 0.98	
Accuracy	0.875	0.686
Balanced Accuracy	0.552	0.789
Precision	0.883	0.984
Recall	0.988	0.649
Specificity	0.116	0.929
MCC	0.222	0.392
False Positives	99	8
False Negatives	9	264

This experiment shows that the practical usefulness of the model depends strongly on the selected threshold and deployment context. A lower threshold favors phishing detection but blocks many legitimate websites. A higher threshold reduces disruption for legitimate users but allows more phishing websites to pass through.

2.4 Limitations of the Feature Representation

The source describes 17 extracted values, but the textual Domain field is removed before training. Therefore, the classifiers receive 16 numerical predictors.

The processed dataset contains only 771 unique numerical feature vectors among 10,000 rows. A total of 109 numerical feature vectors are associated with both labels, and 6,382 rows belong to these conflicting patterns.

This means that different websites can appear identical to the model while receiving different target labels. A deterministic classifier cannot distinguish observations that have exactly the same input values.

A descriptive majority-vote analysis showed that a deterministic classifier using only the supplied 16 numerical predictors could correctly classify at most approximately 86.77% of the rows in this specific processed dataset if every repeated feature vector is assigned its most common label.

This value should not be interpreted as a universal limit for phishing detection. A richer feature representation could distinguish observations that currently appear identical. However, the result suggests that improving the classifier alone may provide limited benefits unless the extracted features are also improved.

2.5 Dataset-Specific Feature Patterns

The feature analysis identified relationships that may reflect the data-collection process rather than universal phishing behavior.

The most influential predictor was URL_Length. The source converts URL length into a binary indicator:

- 0: fewer than 54 characters.
- 1: at least 54 characters.

However, all 5,000 legitimate observations received the value 1, while only 54.68% of phishing observations received this value. Legitimate URLs also had a higher average URL depth than phishing URLs.

Feature	Legitimate Observations	Phishing Observations
URL_Length = 1	100.00%	54.68%
Average URL_Depth	3.327	2.817

Long and deep URLs are therefore associated more strongly with legitimate observations in this dataset. This is unexpected because long or complex URLs are commonly treated as suspicious indicators.

A plausible explanation is that the two classes were collected from different sources and may have different structural characteristics. The model may partly learn dataset-specific collection artifacts rather than general phishing behavior.

This interpretation is supported by the error analysis. Missed phishing observations were more likely to have longer and deeper URL patterns, making them resemble the legitimate class according to the supplied representation.

2.6 Reproducibility and Feature-Extraction Limitations

The supplied processed CSV file can be loaded and used for model training. However, controlled tests of the standalone feature-extraction script revealed several implementation limitations:

Issue	Observed Limitation
Schema mismatch	The script declares 18 fields but returns only 16 numerical values
IP-address detection	The function detects a bare IP address but fails to detect an IP-based hostname inside a complete URL
URL-length transformation	Raw URL length is reduced to a binary threshold, discarding information
Iframe detection	Plain HTML and HTML containing an iframe produced the same output
Right-click detection	The suspicious JavaScript pattern receives the value 0, while a normal script receives 1
WHOIS handling	The standalone script comments out the WHOIS import but silently converts the resulting failure into fallback feature values

These issues do not prevent training on the supplied CSV file. However, they limit the reliability of the pipeline when processing new URLs. A real phishing detector depends not only on the quality of the classifier but also on the correctness and consistency of the feature-extraction stage.

The processed dataset also does not preserve collection timestamps or the complete original URLs. Consequently, the analysis cannot evaluate temporal generalization, inspect complete URL paths, analyze redirect chains, or determine whether unusual examples reflect labeling inconsistencies.

2.7 Are the Author's Conclusions Justified?

The author's main model-comparison result is supported under the original evaluation setting: XGBoost achieves approximately 86% accuracy and performs better than the simpler reproduced tree-based models.

However, the broader practical conclusion requires qualification. The original random split contains extensive train-test contamination, the apparent class balance depends heavily on duplicated legitimate observations, and the feature representation is coarse. The standalone extraction script also requires corrections before it can be considered reliable for inference on new URLs.

Therefore, the project should be viewed as a useful educational baseline rather than as a deployment-ready phishing-detection system.

The main contradiction identified by this reproduction study is not that the reported XGBoost accuracy is false. The score is approximately reproducible. The issue is that the reported accuracy alone does not provide sufficient evidence that the proposed detector will generalize reliably to unseen websites or operate effectively in a real-world environment.

3. Feature Engineering Analysis

3.1 Initial Tree-Based Feature-Importance Analysis

The original model-training pipeline uses 16 numerical predictors after removing the textual Domain column. To examine which features contribute most strongly to the tree-based baseline models, feature-importance values were extracted from the Decision Tree and Random Forest classifiers.

The Random Forest relied primarily on a small number of address-bar-based features. The most important predictor was URL_Length, with an importance value of approximately 0.589. This was followed by Prefix/Suffix with approximately 0.171, URL_Depth with approximately 0.126, and Have_At with approximately 0.035. Together, these four features accounted for approximately 92.1% of the total Random Forest importance.

The meaning of URL_Length requires clarification. The source does not preserve the original numerical length of each URL. Instead, URLs containing fewer than 54 characters receive the value 0, while URLs containing at least 54 characters receive the value 1. Therefore, the model relies heavily on whether each URL crosses a fixed threshold rather than on the complete length information.

The most influential predictors can be extracted directly from the URL without contacting the website. This is relevant from a deployment perspective because URL-based indicators can be calculated quickly. In contrast, domain-based and HTML-or-JavaScript-based features generally received lower importance values. Some of these features require external services, WHOIS queries, or webpage retrieval, which may introduce latency and reliability issues.

These results should be interpreted cautiously. Tree-based importance values indicate how the fitted model used each feature, but they do not prove causation. Low importance may result from redundancy, limited variation, extraction issues, or the restricted tree depth. Later experiments compare the original full feature set with a lightweight URL-only feature set to evaluate whether the more complex domain-based and webpage-content features provide measurable value.

3.2 Preliminary Exploratory Analysis of the Extracted Features

The numerical predictors were examined to identify constant columns, duplicated columns, and differences between legitimate and phishing observations. No completely constant columns were found, and no two numerical features contained identical values across all rows. However, several features were highly imbalanced and provided very limited variation.

The strongest differences between the two classes were observed for URL_Depth, URL_Length, and Prefix/Suffix. The Prefix/Suffix feature behaved as expected: it was present in approximately 0.5% of legitimate observations and 18.1% of phishing observations. The Have_At, TinyURL, and Have_IP indicators were also more common among phishing samples.

The behavior of URL_Length was more surprising. The source converts the original URL length into a binary feature: URLs containing fewer than 54 characters receive the value 0, while URLs containing at least 54 characters receive the value 1. All 5,000 legitimate observations received the value 1, whereas only approximately 54.7% of the phishing observations received this value. Therefore, in the supplied dataset, the long-URL indicator is more strongly associated with legitimate websites, even though the source presents long URLs as a suspicious characteristic.

A similar pattern was observed for URL_Depth. Legitimate observations had an average depth of approximately 3.327, compared with approximately 2.817 for phishing observations.

These results suggest that the classifier may partly learn characteristics of the data-collection process rather than general indicators of phishing behavior. Because the legitimate and phishing URLs were collected from different sources, some extracted features may distinguish the source datasets instead of distinguishing malicious and legitimate websites in a generalizable manner. This interpretation is a hypothesis that requires further testing.

Several predictors were almost constant. The https_Domain feature received the value 1 in only 2 out of 10,000 rows, while the Right_Click feature received the value 1 in 9,993 out of 10,000 rows. Such features are unlikely to contribute substantial predictive information in their current form.

The feature-extraction implementation also requires further review. Preliminary inspection identified possible inconsistencies between the written explanations and the implemented logic for some predictors, including IP-address detection, right-click detection, iframe detection, and web-traffic analysis. These issues are examined in the reproducibility analysis.

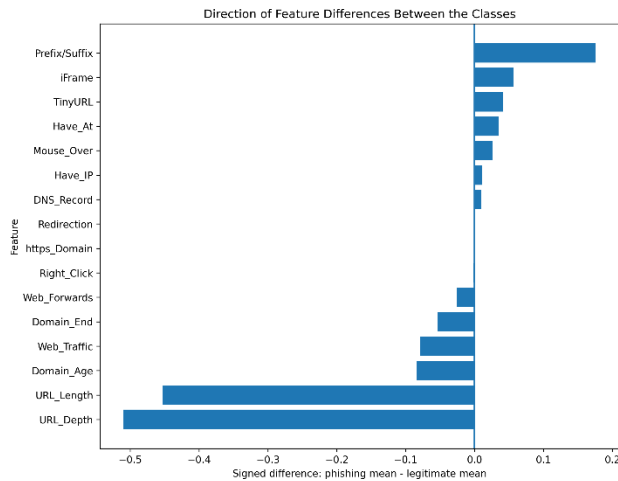


Figure : Signed differences between the mean feature values of phishing and legitimate observations. Positive values indicate features that are more prevalent among phishing samples, while negative values indicate features that are more prevalent among legitimate samples.

3.3 Direction and Prevalence of the Extracted Features

Feature importance identifies the predictors used most heavily by a fitted model, but it does not indicate the direction of the relationship between a feature and the target label. Therefore, the prevalence and average value of each predictor were compared separately for legitimate and phishing observations.

Several features behaved consistently with their intended cybersecurity interpretation. The clearest example was Prefix/Suffix, which indicates whether a hyphen appears in the domain. This feature was present in only 0.52% of legitimate observations but in 18.12% of phishing observations. The iFrame, TinyURL, Have_At, and Have_IP indicators were also more common among phishing samples.

Feature	Legitimate value or prevalence	Phishing value or prevalence	Difference
Prefix/Suffix	0.52%	18.12%	+17.60 percentage points
iFrame	6.26%	11.92%	+5.66 percentage points
TinyURL	6.94%	11.12%	+4.18 percentage points
Have_At	0.50%	4.02%	+3.52 percentage points
URL_Length	100.00%	54.68%	-45.32 percentage points
URL_Depth	3.327	2.817	-0.510

The behavior of URL_Length was unexpected. The original feature-extraction logic assigns the value 1 when a URL contains at least 54 characters. Every legitimate observation received this value, while only 54.68% of phishing observations received it. The legitimate URLs also had a higher average URL depth than the phishing URLs.

These findings suggest that some predictors may capture differences between the data-collection sources rather than general phishing behavior. The legitimate and phishing URLs were obtained from different sources, which may introduce collection artifacts. Consequently, a model may achieve strong performance within this dataset while generalizing less effectively to URLs collected under different conditions.

The analysis also identified near-constant features. The https_Domain feature received the value 0 in 99.98% of the observations, while Right_Click received the value 1 in 99.93% of the observations. The Have_IP feature received the value 0 in 99.45% of the observations. Near-constant features generally provide limited information, although rare cybersecurity indicators should not be removed automatically without considering their practical meaning.

The implementation of these features is examined further in the reproducibility analysis.

3.4 Outlier, Correlation, and Redundancy Analysis

The supplied processed dataset does not contain a timestamp or collection-date column. Therefore, temporal analysis could not be performed. This limits the ability to assess whether the trained classifiers generalize to newer phishing campaigns or whether the relationship between features and labels changes over time.

Most predictors are binary indicators, so conventional numerical outlier analysis is not meaningful for those columns. The URL_Depth feature was analyzed separately because it is a numerical count variable. Its median value is 3, while its interquartile range extends from 2 to 4. Using the standard IQR rule, values above 7 were identified as potential outliers. A total of 357 observations exceeded this threshold, and the maximum depth was 20. These observations were retained because unusually deep URLs may be valid and potentially informative in a cybersecurity context.

Spearman correlation was selected for exploratory correlation analysis. Spearman correlation measures the strength and direction of a monotonic relationship using ranked values. It is suitable for this dataset because the predictors are primarily binary or ordinal indicators, while URL_Depth is a non-normally distributed count variable containing outliers. Compared with Pearson correlation, Spearman correlation is less sensitive to extreme values and does not assume a linear relationship. However, the results must still be interpreted cautiously because many tied values occur in binary variables, and correlation does not imply causation.

The strongest relationship with the target label was observed for URL_Length, with a Spearman correlation of approximately -0.541 . The negative sign indicates that the

long-URL indicator was more common among legitimate observations. The Prefix/Suffix feature showed the strongest positive correlation with the phishing label, with a value of approximately 0.303. The remaining correlations with the target were relatively weak.

The predictor-level analysis also identified potential redundancy. The correlation between iFrame and Mouse_Over was approximately 0.807, while the correlation between Mouse_Over and Web_Forwards was approximately 0.750. These variables are not identical, but they may capture overlapping webpage or JavaScript behavior. A lightweight feature-subset experiment is later used to examine whether all extracted predictors are necessary.

3.5 Feature-Engineering Process and Preprocessing Decisions

Feature engineering was performed by the original source before the processed dataset was generated. The source transforms raw URLs, domain information, and webpage behavior into numerical predictors suitable for machine-learning models.

The supplied processed dataset contains the textual Domain field, 16 numerical predictors, and the binary target variable Label. The original model-training notebook removes Domain before training. Therefore, the classifiers receive the following 16 numerical predictors:

Category	Numerical Predictors
Address-bar-based features	Have_IP, Have_At, URL_Length, URL_Depth, Redirection, https_Domain, TinyURL, Prefix/Suffix
Domain-based features	DNS_Record, Web_Traffic, Domain_Age, Domain_End
HTML-and-JavaScript-based features	iFrame, Mouse_Over, Right_Click, Web_Forwards

Most predictors are binary indicators. URL_Depth is a numerical count feature. The URL_Length feature is also binary because the original source transforms the raw URL length using a fixed threshold:

- 0: the URL contains fewer than 54 characters.
- 1: the URL contains at least 54 characters.

This transformation reduces dimensionality and simplifies interpretation, but it also discards information. For example, URLs containing 54 and 150 characters receive the same value even though their structures may differ substantially.

The textual Domain field was not encoded. One-hot encoding would generate a high-dimensional sparse representation containing thousands of domain-specific columns. It could also encourage memorization of domains observed during training rather than

learning patterns that generalize to unseen websites. The field was therefore retained only for duplicate analysis, domain-aware splitting, and error interpretation.

Normalization and feature scaling were not applied. The evaluated classifiers are Decision Tree, Random Forest, and XGBoost models. These tree-based methods use threshold-based splits and generally do not require standardization or min-max scaling. Scaling would be more relevant for distance-based or gradient-based models such as K-Nearest Neighbors, Support Vector Machines, or Logistic Regression.

No logarithmic transformation, Box–Cox transformation, aggregation, target encoding, or dimensionality-reduction method was applied. Techniques such as PCA were not necessary because the dataset contains only 16 numerical predictors and their cybersecurity interpretation is important. Replacing these predictors with abstract principal components would reduce interpretability without addressing the main identified problems: duplicated observations, coarse feature vectors, and unreliable extraction logic.

3.6 Feature Selection and Redundancy

Several methods were used to investigate whether all predictors were necessary:

1. Constant-feature analysis.
2. Near-constant-feature analysis.
3. Checks for identical feature columns.
4. Spearman-correlation analysis.
5. Tree-based feature importance.
6. A controlled comparison between the full feature set and a lightweight URL-only subset.

No fully constant or duplicated numerical feature columns were found. However, several near-constant predictors were identified. For example, `https_Domain` received the value 0 in 99.98% of observations, while `Right_Click` received the value 1 in 99.93% of observations. Such predictors provide limited variation in their current form.

The correlation analysis also identified potential redundancy. The correlation between `iFrame` and `Mouse_Over` was approximately 0.807, while the correlation between `Mouse_Over` and `Web_Forwards` was approximately 0.750. These features are not identical, but they may capture overlapping webpage or JavaScript behavior.

To test whether the complex features were necessary, the full 16-feature XGBoost model was compared with a lightweight model using only eight predictors extracted directly from the URL string:

`Have_IP`, `Have_At`, `URL_Length`, `URL_Depth`, `Redirection`, `https_Domain`, `TinyURL`, and `Prefix/Suffix`.

The comparison was performed using the domain-aware split.

Metric	Full 16-Feature XGBoost	URL-Only XGBoost
Accuracy	0.875	0.868
Balanced Accuracy	0.552	0.499
Precision	0.883	0.870
Recall	0.988	0.997
Specificity	0.116	0.000
MCC	0.222	-0.019

The URL-only model appears similar when accuracy is considered alone. However, it failed to correctly allow any legitimate website at the default threshold. The full representation achieved a higher balanced accuracy and MCC, indicating that the additional domain-based and webpage-content features provide limited but measurable value.

Potential redundancy should be addressed cautiously. Features should not be removed solely because they are rare or correlated. A rare cybersecurity indicator may still be useful when it occurs. A suitable approach would be to correct the extraction logic first and then use controlled ablation experiments with domain-aware validation to determine whether removing near-constant or correlated predictors improves generalization, reliability, and runtime.

3.7 Meaningfulness of the Feature Engineering and Suggested Improvements

The original feature-engineering approach is meaningful as an educational baseline. Several predictors have clear cybersecurity interpretations. For example:

- A URL containing an IP-based hostname may be suspicious because legitimate services usually use recognizable domain names.
- The @ symbol may obscure the real destination of a URL.
- URL-shortening services can hide the final destination.
- A hyphenated domain may imitate a legitimate service.
- Domain-age and DNS-related information may help identify newly created malicious infrastructure.

The approach also has practical advantages. Address-bar-based features can be extracted quickly without retrieving webpage content. This is useful for browser extensions or real-time filtering systems.

However, the analysis identified limitations. Some predictors provide little variation, some extraction functions appear unreliable, and the most influential feature may reflect dataset-specific collection artifacts. The binary URL_Length feature is particularly important: all legitimate observations crossed the 54-character threshold, while only 54.68% of phishing observations did. As a result, the model may partially learn differences between the source datasets rather than universal phishing behavior.

Several additional features could improve the representation:

Suggested Feature	Potential Benefit
Raw URL length	Preserves information discarded by the binary threshold
Number of subdomains	Helps identify misleading or deeply nested hostnames
Ratio of digits and special characters	Captures unusual lexical patterns
URL entropy	Measures randomness that may indicate generated or obfuscated URLs
Top-level domain category	Identifies suspicious or uncommon domain endings
Number of query parameters	Captures complex or potentially obfuscated URLs
Character-level n-grams	Learns lexical patterns without relying only on handcrafted thresholds
Properly extracted hostname IP indicator	Corrects the current IP-detection limitation
Correctly parsed HTML features	Improves the reliability of iframe and script-related indicators
TLS certificate information	Adds evidence about certificate validity and age
Correctly extracted domain-age and DNS features	Improves domain-reputation analysis
Collection timestamp	Enables temporal validation against newer phishing campaigns
Complete original URL	Enables more reliable error analysis and feature creation

Additional features should be evaluated with domain-aware and time-aware validation when possible. Otherwise, a more complex feature set may appear to improve accuracy while still failing to generalize to new phishing campaigns.

4. Reproducibility Analysis

4.1 Executability of the Supplied Model-Training Pipeline

The processed dataset supplied by the original source can be loaded successfully from the file `5.urldata.csv`. The file contains 10,000 observations and 18 columns: the textual Domain field, 16 numerical predictors, and the binary target variable Label.

The model-training experiment can be reproduced using the supplied processed CSV file. Decision Tree, Random Forest, and XGBoost classifiers were trained successfully. Under an evaluation setting comparable to the original source, the reproduced XGBoost model achieved approximately 86.3% testing accuracy, which is close to the reported value of 86.4%.

Therefore, the core model-training result is approximately reproducible when the prepared dataset is used.

However, the original experiment is not fully deterministic. The source shuffles the dataset without defining a fixed random seed, and some model configurations also omit explicit seeds. As a result, repeated executions may produce slightly different train-test splits and scores.

To address this issue, a controlled version of the experiment was created using fixed random seeds and stratification by the target label. Under this deterministic split, XGBoost achieved approximately 86.6% testing accuracy. This confirms that the main baseline result is stable enough to reproduce approximately, even though the original implementation does not guarantee identical outputs across executions.

4.2 Required Files and Dependencies

The reproduction study uses the following supplied files:

File	Purpose
<code>5.urldata.csv</code>	Processed dataset used for model training and evaluation
<code>URLFeatureExtraction.py</code>	Standalone script intended to extract numerical predictors from new URLs
Original source notebooks and supporting files	Reference implementation for preprocessing and model comparison

The model-training notebook requires the following Python libraries:

Library	Purpose
pandas	Dataset loading and tabular analysis
numpy	Numerical operations
matplotlib and seaborn	Visualizations
scikit-learn	Data splitting, classifiers, and evaluation metrics
xgboost	XGBoost classifier

The standalone feature-extraction script has additional requirements. It depends on network access and external lookups for some domain-related and webpage-content features. It also attempts to use WHOIS functionality. These dependencies make end-to-end reproduction more fragile than reproduction of the model-training notebook alone.

For the final submitted repository, the required files should be stored using consistent relative paths and accompanied by execution instructions and a pinned requirements.txt file.

4.3 Hidden and Incomplete Preprocessing Steps

The supplied processed CSV file can be used directly for model training. However, the complete transformation from raw URLs to the processed dataset is not fully reproducible using the standalone script in its current form.

The source documentation describes 17 extracted values. The processed dataset contains the textual Domain field, 16 numerical predictors, and the target variable. Before training, the textual field is removed. Therefore, the classifiers actually receive 16 numerical predictors.

The standalone extraction script is not fully equivalent to the preprocessing workflow used to generate the supplied CSV file. Its featureExtraction(url) function returns 16 numerical values, while its declared column-name list contains 18 fields: Domain, the 16 numerical predictors, and Label. Attempting to construct a DataFrame using the declared column list and the returned values produces a ValueError.

The standalone script also does not expose the getDomain function used during dataset creation and does not import the WHOIS library, even though it attempts to call WHOIS functionality. Consequently, additional code and dependency corrections are required to regenerate the complete processed dataset from raw URLs.

This means that the supplied CSV is sufficient for reproducing the classification experiment, but the preprocessing stage contains incomplete or hidden details that prevent a clean end-to-end reconstruction.

4.4 Controlled Audit of the Feature-Extraction Script

Selected extraction functions were tested using controlled examples that do not depend on live webpages.

Feature or Component	Controlled Finding
Schema consistency	The script declares 18 columns but returns only 16 numerical values
IP-address detection	A bare IP address is detected, but IP-based hostnames inside complete URLs are missed
URL_Length	Raw length is reduced to a binary threshold; URLs of length 54 and 80 receive the same value
Iframe detection	Plain HTML and HTML containing an <iframe> tag produced the same result
Right-click detection	A normal script receives the value 1, while the suspicious pattern <code>event.button == 2</code> receives 0
WHOIS handling	A missing WHOIS dependency is silently converted into fallback feature values

The IP-address issue occurs because the script attempts to parse the complete URL as an IP address rather than extracting and inspecting the hostname. For example, the bare value 192.168.1.10 is detected correctly, while `http://192.168.1.10/login` is not.

The iframe-detection logic also appears unreliable. The implemented regular expression uses square brackets, which create a character class rather than matching the complete HTML tag.

The WHOIS handling is particularly important. The standalone script comments out the WHOIS import but continues to call WHOIS functionality inside a broad exception handler. When the lookup fails, the script silently assigns fallback values to `DNS_Record`, `Domain_Age`, and `Domain_End`. These values may therefore reflect a dependency failure rather than the real properties of the tested website.

These issues do not prevent training on the supplied processed CSV. However, they reduce confidence in the reliability of the pipeline when it is applied to new URLs.

4.5 Data-Independence Limitation

A deterministic controlled split was created using a fixed seed and stratification. This improves reproducibility because the same train-test split can be regenerated consistently.

However, fixed seeds do not solve the data-independence problem. In the controlled random split, 1,272 out of 2,000 testing rows were identical to complete rows in the training set, and 1,427 testing observations used domains that also appeared during training.

This overlap is caused primarily by duplicated observations and repeated domains in the dataset rather than by the absence of a fixed seed. The issue is examined in greater detail in the Critical Evaluation section.

4.6 Overall Reproducibility Assessment

The work is partially reproducible.

The supplied processed CSV file allows the core classification experiment to run successfully. The reported XGBoost result can be reproduced approximately, and a deterministic controlled implementation produces a similar score.

However, the full end-to-end pipeline is not reliably reproducible in its current form. The preprocessing workflow cannot be reconstructed cleanly from the standalone script without additional corrections. Some external dependencies are missing or fragile, several extraction functions behave inconsistently under controlled tests, and external web-based lookups may change over time or fail unexpectedly.

The original source is therefore sufficient for reproducing the baseline model-training experiment, but it is not sufficient for confidently regenerating the dataset or deploying a reliable phishing detector on new URLs without further engineering work.

5. Experimental Results

5.1 Experimental Overview

The experimental analysis was divided into several stages. The first stage reproduced the original model-training methodology as closely as possible. Later stages introduced controlled modifications to evaluate whether the reported results remained reliable under stricter conditions.

Experiment	Modification Introduced	Main Purpose
Original-style baseline reproduction	Original random 80–20 split retained	Reproduce the reported baseline results
Controlled random split	Fixed random seeds and stratification added	Improve determinism while preserving a methodology comparable to the source
Deduplicated random split	Exact duplicated rows removed before splitting	Measure the effect of repeated observations
Domain-aware split	Each domain assigned entirely to training or testing	Evaluate generalization to unseen domains
Feature-subset comparison	Full 16-feature model compared with an 8-feature URL-only model	Measure the contribution of domain-based and webpage-content features
Threshold analysis	Threshold selected using a separate domain-aware validation set	Examine the trade-off between False Positives and False Negatives

Three supervised classifiers were trained and compared:

- Decision Tree
- Random Forest
- XGBoost

The original-style results are reported using accuracy, precision, recall, (F₁), (F₂), MCC, ROC-AUC, Average Precision, and confusion-matrix values. Balanced accuracy and specificity are emphasized in the stricter experiments because duplicate removal creates substantial class imbalance.

5.2 Baseline Decision Tree Reproduction

The first reproduced model was a Decision Tree classifier with a maximum depth of 5, matching the configuration used in the original notebook. The model was trained and evaluated using the original random 80–20 split.

The reproduced Decision Tree achieved a training accuracy of 81.2% and a testing accuracy of 82.0%. The testing results are summarized below.

Metric	Value
---------------	--------------

Accuracy	0.820
----------	-------

Precision	0.968
-----------	-------

Recall	0.653
--------	-------

F1-score	0.780
----------	-------

F2-score	0.699
----------	-------

MCC	0.673
-----	-------

ROC-AUC	0.855
---------	-------

PR-AUC	0.860
--------	-------

The confusion matrix contained 1,001 True Negatives, 21 False Positives, 339 False Negatives, and 639 True Positives.

The model achieved high precision: when it classified a website as phishing, its prediction was usually correct. It also produced relatively few False Positives, meaning that only a small number of legitimate websites were incorrectly blocked. However, the recall was substantially lower. The model failed to detect 339 out of 978 phishing observations in the testing set, corresponding to approximately 34.7% of the phishing samples.

From a cybersecurity perspective, this trade-off is important. A low False Positive rate reduces unnecessary disruption for users, but a detector that misses approximately one-third of phishing websites would be insufficient as a standalone defensive mechanism.

The original notebook reports a Decision Tree testing accuracy of approximately 82.6%, while the reproduction obtained 82.0%. The small difference is likely caused by the non-deterministic shuffle performed before the train-test split. Because the shuffle does not use a fixed random seed, the exact training and testing samples may differ between executions.

These baseline results must also be interpreted cautiously because the original random split contains duplicated observations and repeated domains across the training and testing sets. Later experiments evaluate performance under stricter conditions.

5.3 Baseline Random Forest Reproduction

The second reproduced model was a Random Forest classifier with a maximum tree depth of 5, matching the configuration used by the original notebook. The classifier

used 100 trees, which is the current default value in scikit-learn. The original source does not specify a fixed random seed for the Random Forest model.

The reproduced Random Forest achieved a training accuracy of 81.7% and a testing accuracy of 82.2%. The testing results are summarized below.

Metric	Value
Accuracy	0.822
Precision	0.973
Recall	0.653
F1-score	0.782
F2-score	0.699
MCC	0.677
ROC-AUC	0.885
PR-AUC	0.910

The confusion matrix contained 1,004 True Negatives, 18 False Positives, 339 False Negatives, and 639 True Positives.

Compared with the Decision Tree baseline, the Random Forest reduced the number of False Positives from 21 to 18. However, it did not reduce the number of False Negatives. Both models missed 339 phishing observations. As a result, the recall remained 65.3%.

The Random Forest produced a more noticeable improvement in ROC-AUC and PR-AUC. This suggests that its probability scores rank suspicious websites more effectively, even though the default classification threshold does not convert this improvement into a substantial increase in the number of detected phishing samples. A later experiment examines whether threshold adjustment can improve the trade-off between False Positives and False Negatives.

The original notebook reports a Random Forest testing accuracy of approximately 83.4%, while the reproduced run achieved 82.2%. This variation is expected because the original dataset shuffle and the Random Forest classifier do not use fixed random seeds. Therefore, the original experiment is approximately reproducible but not fully deterministic.

5.4 Baseline XGBoost Reproduction

The source reports that XGBoost achieved the highest performance among the evaluated models, with a testing accuracy of approximately 86.4%. To evaluate this claim, an XGBoost classifier was trained using the configuration specified in the original notebook: a learning rate of 0.4 and a maximum tree depth of 7. Other parameters were inherited from the installed XGBoost environment because they were not explicitly defined by the source.

The reproduced XGBoost model achieved a training accuracy of 86.7% and a testing accuracy of 86.3%. This result is very close to the value reported by the source and therefore supports the claim that XGBoost performs better than the simpler Decision Tree and Random Forest baselines under the original random-split methodology.

Metric	Value
--------	-------

Accuracy	0.863
----------	-------

Precision	0.912
-----------	-------

Recall	0.798
--------	-------

F1-score	0.851
----------	-------

F2-score	0.819
----------	-------

MCC	0.731
-----	-------

ROC-AUC	0.918
---------	-------

PR-AUC	0.936
--------	-------

The confusion matrix contained 943 True Negatives, 76 False Positives, 198 False Negatives, and 783 True Positives.

Compared with the Random Forest baseline, XGBoost detected 148 additional phishing observations but incorrectly blocked 65 additional legitimate observations. This demonstrates an important cybersecurity trade-off. XGBoost is more effective at detecting malicious websites, but its increased sensitivity also produces more disruption for legitimate users.

The reproduced model still failed to detect approximately 20.2% of the phishing observations. In addition, approximately 7.5% of the legitimate observations were incorrectly classified as phishing. Whether this trade-off is acceptable depends on the deployment context. For example, a browser warning system may tolerate more False Positives than a system that automatically blocks access to websites.

The reproduced score should be interpreted as a baseline rather than as a realistic estimate of deployment performance. The original random split contains substantial

overlap between training and testing observations. Furthermore, the original preprocessing shuffle and several model seeds are not fixed, so repeated executions may produce slightly different results.

5.5 Controlled Random Split with Fixed Seeds

The original source does not define a fixed seed when shuffling the dataset, and some model configurations also omit explicit random seeds. As a result, repeated executions may produce slightly different training and testing samples.

A controlled version of the random-split experiment was therefore created using:

- `random_state=42`;
- stratification by the target label;
- explicit model seeds;
- an explicit number of XGBoost boosting rounds.

The controlled split retained the complete processed dataset, including duplicated observations, so that the results remained comparable with the original methodology. The training set contained 4,000 legitimate and 4,000 phishing observations, while the testing set contained 1,000 observations from each class.

Model	Accuracy	Precision	Recall	F1-score	MCC	ROC-AUC
Decision Tree	0.810	0.973	0.638	0.771	0.660	0.851
Random Forest	0.818	0.980	0.649	0.781	0.676	0.887
XGBoost	0.866	0.928	0.795	0.856	0.741	0.926

XGBoost remained the strongest model. Its accuracy of 86.6% is close to both the original reported result of 86.4% and the reproduced original-style result of 86.3%.

Fixing the seeds improves reproducibility, but it does not solve the independence problem. The controlled random split still contained 1,272 testing rows that were identical to complete training rows and 1,427 testing observations whose domains appeared during training. These limitations motivated the deduplicated and domain-aware experiments.

5.6 Deduplicated Random-Split Experiment

To evaluate the effect of repeated observations, exact duplicated rows were removed before performing a new stratified train-test split. The dataset decreased from 10,000 rows to 4,374 unique complete observations, meaning that 5,626 duplicate rows were removed.

Duplicate removal substantially changed the class distribution. In the original dataset, the two classes each contained 5,000 rows. After deduplication, only 529 legitimate observations remained, compared with 3,845 phishing observations. Legitimate websites therefore represented only 12.09% of the deduplicated dataset.

Dataset version	Legitimate observations	Phishing observations
Original dataset	5,000	5,000
Deduplicated dataset	529	3,845

The results demonstrate why accuracy alone is insufficient for evaluating the proposed detector.

Model	Accuracy	Balanced Accuracy	Precision	Recall	Specificity	MCC
Decision Tree	0.901	0.598	0.900	0.997	0.198	0.399
Random Forest	0.879	0.500	0.879	1.000	0.000	0.000
XGBoost	0.887	0.622	0.907	0.971	0.274	0.341

The Random Forest classified every test observation as phishing. Because phishing observations represented approximately 87.91% of the deduplicated dataset, the model still achieved an accuracy of 87.9%. However, its specificity and MCC were both equal to zero. It failed to correctly identify even one legitimate website.

XGBoost produced the strongest balanced accuracy among the three models, but its specificity remained low. It correctly classified only 29 out of 106 legitimate websites and incorrectly blocked the remaining 77 legitimate observations. Therefore, its high recall must be interpreted together with its False Positive rate.

This experiment shows that the apparent balance of the original dataset depended heavily on duplicated legitimate observations. After exact duplicates were removed, accuracy became particularly misleading. Metrics such as balanced accuracy, specificity, and MCC provide a more meaningful evaluation of model behavior under class imbalance.

Exact complete-row overlap between training and testing was eliminated. However, 232 out of 875 testing observations still used a domain value that appeared in training. A domain-aware experiment is therefore required to evaluate generalization to previously unseen domains.

5.7 Domain-Aware Split Experiment

The deduplicated random split eliminated exact duplicated rows across training and testing, but it still allowed the same domain to appear in both sets. To evaluate generalization more rigorously, a domain-aware split was created using stratified group splitting. Each domain was assigned entirely to either the training set or the testing set.

The resulting training set contained 3,509 observations, while the testing set contained 865 observations. No complete row and no domain appeared in both sets.

Overlap Check	Result
Exact complete-row overlap	0
Test rows whose domain appeared in training	0
Domains appearing in both sets	0
Test rows sharing a numerical feature vector with training	792

Although domain overlap was eliminated, 792 out of 865 testing observations shared their numerical feature vector with at least one training observation. This indicates that the 16 extracted predictors form a relatively coarse representation: different URLs from different domains may still appear identical to the classifier.

The domain-aware results are summarized below.

Model	Accuracy	Balanced Accuracy	Precision	Recall	Specificity	MCC
Decision Tree	0.877	0.534	0.878	0.997	0.071	0.216
Random Forest	0.871	0.500	0.871	1.000	0.000	0.000
XGBoost	0.875	0.552	0.883	0.988	0.116	0.222

The accuracy values remain high, but they are misleading because approximately 87% of the testing observations are phishing samples. The Random Forest classified every testing observation as phishing and still achieved an accuracy of 87.1%. Its balanced accuracy was 0.500, its specificity was 0.000, and its MCC was 0.000.

XGBoost achieved the strongest balanced accuracy and MCC among the evaluated models. It detected 744 out of 753 phishing observations, corresponding to a recall of approximately 98.8%. However, it correctly allowed only 13 out of 112 legitimate websites. The remaining 99 legitimate observations were incorrectly blocked, corresponding to a False Positive rate of approximately 88.4%.

These results show that the original reported accuracy is reproducible under a random split but does not provide sufficient evidence that the detector generalizes reliably to previously unseen domains. A practical phishing-detection system must balance the detection of malicious URLs with the ability to avoid blocking legitimate websites.

5.8 Full Feature Set versus URL-Only Features

The full representation contains 16 numerical predictors. Some predictors require domain lookups, external services, or webpage-content retrieval. The reproducibility audit identified limitations in several of these extraction functions.

A lightweight XGBoost model was therefore trained using only eight predictors extracted directly from the URL string:

- Have_IP
- Have_At
- URL_Length
- URL_Depth
- Redirection
- https_Domain
- TinyURL
- Prefix/Suffix

The full and lightweight models were compared using the domain-aware split.

Metric	Full 16-Feature XGBoost	URL-Only XGBoost
Accuracy	0.875	0.868
Balanced Accuracy	0.552	0.499
Precision	0.883	0.870
Recall	0.988	0.997
Specificity	0.116	0.000
MCC	0.222	-0.019
ROC-AUC	0.857	0.860
Average Precision	0.977	0.976

The two models appear similar when accuracy is considered alone. However, the URL-only model failed to correctly allow any legitimate website at the default threshold. Its specificity was 0.000, and its MCC was slightly negative.

The full feature representation also performed poorly at the default threshold, but it retained limited discriminatory value. It correctly allowed 13 legitimate observations and achieved a higher balanced accuracy and MCC.

These results show that the additional domain-based and webpage-content features provide limited but measurable value. However, their extraction logic should be corrected before they are used in a real deployment.

5.9 Threshold Analysis

The default classification threshold of 0.50 produced an unsuitable trade-off under the domain-aware evaluation. Although the full 16-feature XGBoost model achieved an accuracy of 87.5% and detected 98.8% of the phishing observations, it incorrectly blocked 99 out of 112 legitimate websites. Its specificity was only 11.6%.

To improve the operating point, threshold selection was performed using a separate domain-aware validation set. No domain appeared in both the internal training and validation subsets, and the final testing set remained untouched during threshold selection. Matthews Correlation Coefficient (MCC) was used as the primary selection criterion, while balanced accuracy was used as a secondary criterion. MCC was selected because it considers all four entries in the confusion matrix and remains informative under class imbalance.

The selected threshold for the full 16-feature XGBoost model was 0.98.

Metric	Default Threshold: 0.50	Validation-Selected Threshold: 0.98
Accuracy	0.875	0.686
Balanced Accuracy	0.552	0.789
Precision	0.883	0.984
Recall	0.988	0.649
Specificity	0.116	0.929
False Positive Rate	0.884	0.071
MCC	0.222	0.392
True Negatives	13	104
False Positives	99	8
False Negatives	9	264
True Positives	744	489

The tuned threshold substantially reduced the number of incorrectly blocked legitimate websites from 99 to 8. However, the number of missed phishing websites increased

from 9 to 264. This illustrates the practical trade-off between False Positives and False Negatives.

The appropriate threshold depends on the deployment context. A warning system may prioritize recall and tolerate additional alerts, while an automatic blocking mechanism requires substantially higher specificity to avoid disrupting legitimate users. The selected threshold of 0.98 should therefore be interpreted as a balanced operating point for this experiment rather than as a universal recommendation.

A lightweight URL-only XGBoost model was also evaluated. At its validation-selected threshold of 0.99, it achieved a specificity of 97.3% but a lower phishing recall of 58.6%. The full feature set therefore provided limited but measurable additional value.

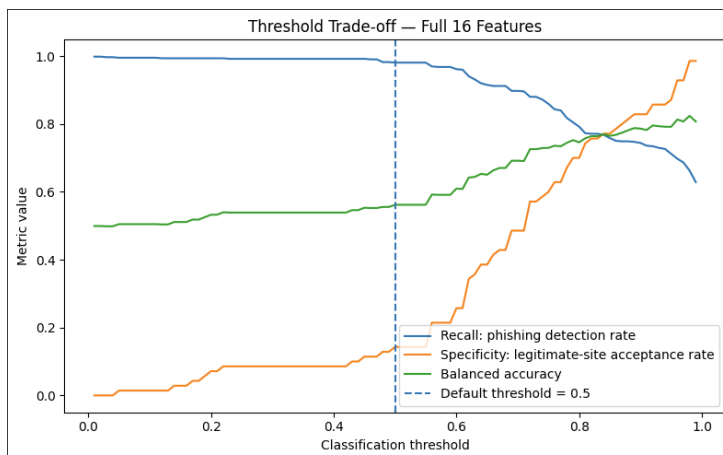


Figure: Trade-off between phishing recall, legitimate-site specificity, and balanced accuracy across XGBoost classification thresholds using the full 16-feature representation.

5.10 Error Analysis

The final error analysis was performed using the full 16-feature XGBoost model under the domain-aware evaluation setting. The classification threshold was set to 0.98, selected using a separate domain-aware validation set. No domain from the final testing set was used during model training or threshold selection.

The resulting confusion matrix contained 104 True Negatives, 8 False Positives, 264 False Negatives, and 489 True Positives.

Outcome	Count
---------	-------

True Negatives	104
----------------	-----

False Positives	8
-----------------	---

False Negatives	264
-----------------	-----

True Positives	489
----------------	-----

The tuned model correctly allowed approximately 92.9% of legitimate websites and detected approximately 64.9% of phishing websites. Compared with the default threshold of 0.50, the selected threshold substantially reduced the number of

incorrectly blocked legitimate websites. However, this improvement came at the cost of allowing additional phishing websites to pass undetected.

5.10.1 False Positives

False Positives represent legitimate websites that are incorrectly blocked. In a practical deployment, these errors may prevent users from accessing valid services, interrupt business operations, and reduce trust in the security system.

The False Positive analysis identified several notable patterns. Compared with correctly allowed legitimate websites, False Positives were more likely to receive lower traffic-related and domain-age-related feature values. They were also more likely to contain a hyphen in the domain, represented by the Prefix/Suffix feature.

Feature	False Positive Mean – True Negative Mean
---------	--

Web_Traffic	-0.548
-------------	--------

Domain_Age	-0.538
------------	--------

Prefix/Suffix	+0.365
---------------	--------

Domain_End	+0.192
------------	--------

DNS_Record	-0.173
------------	--------

The Prefix/Suffix pattern is intuitive because attackers may use hyphenated domains to imitate legitimate websites. However, legitimate websites can also contain hyphens. This demonstrates that suspicious indicators should not be treated as conclusive evidence in isolation.

5.10.2 False Negatives

False Negatives represent phishing websites that are incorrectly allowed. These errors are particularly important because a user may visit a malicious page, disclose credentials, or interact with harmful content.

Compared with correctly detected phishing rows, the missed phishing observations had substantially greater URL depth and were more likely to cross the long-URL threshold. They were also less likely to contain a hyphen.

Feature	False Negative Mean – True Positive Mean
---------	--

URL_Depth	+2.142
-----------	--------

URL_Length	+0.773
------------	--------

Prefix/Suffix	-0.217
---------------	--------

Web_Traffic	+0.067
-------------	--------

Feature	False Negative Mean – True Positive Mean
---------	--

TinyURL	-0.050
---------	--------

These results align with the earlier exploratory analysis. In the supplied dataset, long and deep URLs are associated more strongly with legitimate observations. Therefore, phishing websites with similar patterns may be incorrectly classified as legitimate. This reinforces the concern that the classifier may partly learn dataset-specific collection artifacts rather than universal phishing characteristics.

5.10.3 False Positive–False Negative Trade-Off

The prediction-score distribution reveals substantial overlap between legitimate and phishing observations. Lowering the threshold increases phishing recall but also increases the number of legitimate websites that are blocked. Increasing the threshold reduces disruption for legitimate users but allows more phishing websites to pass through.

The appropriate operating threshold depends on the deployment context. A browser-warning system may tolerate additional False Positives because users can override the warning. In contrast, an automatic blocking system requires higher specificity to avoid disrupting legitimate activity.

The tuned threshold of 0.98 should therefore be interpreted as a validation-based compromise for this experiment rather than as a universal recommendation.

5.10.4 Limitations of Example-Level Interpretation

The processed dataset retains the extracted domain value but not the complete original URL. Consequently, the analysis cannot inspect the full path, query parameters, redirect chains, or original collection context. Some processed phishing rows contain domains associated with well-known services. These cases should not automatically be interpreted as evidence that the domain itself is malicious. They may reflect malicious paths, redirects, compromised pages, preprocessing loss, or labeling inconsistencies.

Preserving the complete original URLs and collection timestamps would enable a more reliable error analysis.

6. Conclusions

6.1 Key Findings

This project critically evaluated the tutorial-style phishing-detection pipeline proposed in the selected source. The original source uses supervised machine-learning models trained on numerical predictors extracted from URLs, domain information, and webpage behavior. It reports that XGBoost achieves the strongest performance, with a testing accuracy of approximately 86.4%.

The main reported result was reproduced successfully. Under an evaluation setting comparable to the original methodology, the reproduced XGBoost model achieved approximately 86.3% testing accuracy. A deterministic version of the experiment using fixed seeds and stratification achieved approximately 86.6% testing accuracy. These results support the narrow conclusion that XGBoost performs better than the reproduced Decision Tree and Random Forest baselines under the original random-split methodology.

However, the analysis also identified important limitations that affect the interpretation of the reported score.

The processed dataset contains 5,626 exact duplicated rows. In the controlled random split, 63.60% of the testing observations are identical to complete rows already present in the training set, while 71.35% of testing observations use domains that also appear during training. Therefore, the original random split does not provide a strong evaluation of generalization to previously unseen websites.

Removing exact duplicated rows also changes the apparent class distribution substantially. The original dataset contains 5,000 observations from each class, but the deduplicated dataset contains only 529 legitimate observations and 3,845 phishing observations. This shows that the apparent balance of the dataset depends heavily on repeated legitimate rows.

A stricter domain-aware experiment was therefore performed. Each domain was assigned entirely to either the training set or the testing set. Under this evaluation, the default XGBoost threshold produced high phishing recall but incorrectly blocked most legitimate websites. A validation-selected threshold of 0.98 improved the balance between the two classes:

Metric	Default Threshold: 0.50 Validation-Selected Threshold: 0.98	
Accuracy	0.875	0.686
Balanced Accuracy	0.552	0.789
Precision	0.883	0.984
Recall	0.988	0.649

Metric	Default Threshold: 0.50 Validation-Selected Threshold: 0.98	
Specificity	0.116	0.929
MCC	0.222	0.392
False Positives	99	8
False Negatives	9	264

The tuned threshold substantially reduced the number of legitimate websites that were blocked incorrectly. However, it also allowed more phishing websites to pass undetected. This demonstrates that no single threshold is suitable for every deployment scenario.

The feature-engineering analysis also identified limitations in the representation. The model relies heavily on a binary URL-length indicator. In the supplied dataset, this feature is associated more strongly with legitimate observations than with phishing observations. The model may therefore learn collection-specific patterns rather than universal phishing behavior.

The controlled audit of the standalone feature-extraction script also revealed implementation issues, including incomplete IP-address detection, unreliable iframe detection, unclear right-click encoding, a schema mismatch, and silent fallback values when WHOIS functionality is unavailable.

6.2 Lessons Learned

Several broader lessons emerged from this analysis.

First, reproducing a reported accuracy value is not enough to validate a cybersecurity system. The reproduced XGBoost score is close to the original reported score, but its practical meaning changes substantially when duplicated observations and repeated domains are examined.

Second, evaluation methodology is as important as model selection. A random split may produce optimistic results when related observations appear in both training and testing. Domain-aware and time-aware splits are more appropriate when the objective is to measure generalization to new websites or phishing campaigns.

Third, accuracy should not be interpreted in isolation. After duplicate removal, the dataset becomes highly imbalanced. A model that predicts every observation as phishing can still obtain high accuracy while failing completely to recognize legitimate websites. Metrics such as recall, specificity, balanced accuracy, MCC, and the confusion matrix provide a more reliable view of model behavior.

Fourth, the feature-extraction pipeline is as important as the classifier. A strong model cannot compensate reliably for incorrect or inconsistent input features. Feature quality,

preprocessing transparency, and dependency handling must be evaluated alongside model performance.

Finally, the correct classification threshold depends on the deployment context. A browser-warning system, an analyst-triage tool, and an automatic blocking system may require different trade-offs between False Positives and False Negatives.

6.3 Strengths of the Proposed Solution

The proposed solution has several strengths:

1. It addresses an important and practical cybersecurity problem.
2. It uses interpretable predictors with understandable security meaning.
3. Several address-bar-based features can be extracted quickly without retrieving webpage content.
4. The supplied processed CSV file enables approximate reproduction of the main model-training experiment.
5. XGBoost consistently performs better than the simpler reproduced baselines under comparable evaluation settings.
6. The overall workflow is useful as an educational introduction to phishing detection with machine learning.

6.4 Weaknesses and Limitations

The proposed solution also has important weaknesses:

1. The processed dataset contains extensive duplication.
2. The original random split produces substantial train-test contamination.
3. The row-level class balance depends heavily on repeated legitimate observations.
4. The 16-predictor representation is coarse: different websites may receive identical feature vectors.
5. The processed dataset does not preserve the complete original URLs or collection timestamps.
6. Several extraction functions behave inconsistently under controlled tests.
7. Some features depend on external services and network access, which may fail or change over time.
8. The original evaluation emphasizes accuracy without sufficiently examining specificity, class imbalance, or deployment-specific threshold selection.

Because of these limitations, the proposed pipeline should not be treated as a deployment-ready phishing detector in its current form.

6.5 Suggestions for Future Improvements

Future work should improve both the dataset and the feature-extraction pipeline.

Dataset and Evaluation Improvements

1. Remove exact duplicated observations before splitting the data.
2. Preserve the complete original URLs rather than only the extracted domain values.
3. Store collection timestamps and evaluate temporal generalization.
4. Use domain-aware or campaign-aware splits so that related observations do not appear in both training and testing.
5. Evaluate the model on a recent external dataset collected independently from the training data.
6. Report balanced accuracy, specificity, MCC, and confusion matrices in addition to accuracy.

Feature-Engineering Improvements

1. Preserve raw URL length rather than reducing it to a single binary threshold.
2. Extract the number of subdomains, query parameters, digits, and special characters.
3. Add lexical indicators such as URL entropy and character-level n-grams.
4. Correct IP-address detection by inspecting the parsed hostname.
5. Replace the iframe regular expression with proper HTML parsing.
6. Clarify and correct the encoding of JavaScript-related indicators.
7. Handle WHOIS, DNS, and external-service failures explicitly rather than silently assigning fallback values.
8. Consider TLS-certificate information and domain-reputation indicators where reliable data are available.

Deployment Improvements

1. Calibrate the model scores before interpreting them as probabilities.
2. Select classification thresholds according to the intended use case.
3. Consider using multiple risk levels rather than a single automatic block-or-allow decision.
4. Combine the classifier with additional security signals and human review where appropriate.

5. Monitor model performance over time because phishing tactics evolve.

6.6 Final Conclusion

The original XGBoost result is approximately reproducible, and the selected source provides a useful educational baseline for phishing detection with machine learning.

However, the reported accuracy alone does not provide sufficient evidence that the proposed detector will generalize reliably to unseen domains or operate effectively in a real-world environment. The dataset construction, evaluation methodology, feature representation, and standalone extraction script all require improvement.

The proposed system should therefore be viewed as a starting point for further research and engineering rather than as a production-ready phishing-detection solution.

Summing It Up

This project critically evaluated the tutorial-style GitHub project **“Phishing Website Detection by Machine Learning Techniques”** by Shreya Gopal Sundari [1]. The selected source addresses the problem of automatically distinguishing phishing websites from legitimate websites using supervised machine-learning models.

Phishing detection is an important cybersecurity task because malicious websites may imitate legitimate services and deceive users into disclosing login credentials, personal information, or financial details. An effective detector should identify malicious websites while avoiding unnecessary blocking of legitimate websites.

The supplied processed dataset contains 10,000 URL observations:

- 5,000 phishing observations collected from PhishTank [2].
- 5,000 legitimate observations obtained from a University of New Brunswick dataset [3].

The source extracts values describing URL characteristics, domain properties, and webpage behavior. After removing the textual Domain field, the trained classifiers receive 16 numerical predictors. The original project compares several models and reports that XGBoost achieves the highest testing accuracy, approximately 86.4%.

The first stage of this reproduction study evaluated whether the reported result could be reproduced. Decision Tree, Random Forest, and XGBoost classifiers were trained using an evaluation setting comparable to the original source. The reproduced XGBoost model achieved approximately 86.3% testing accuracy, while a deterministic version using fixed seeds and stratification achieved approximately 86.6% testing accuracy.

These results support the author’s narrow model-comparison claim: XGBoost performs better than the reproduced Decision Tree and Random Forest baselines under the original random-split methodology.

However, the analysis identified several limitations that weaken the broader conclusion that the proposed detector would generalize reliably to new websites.

The processed dataset contains 5,626 exact duplicated rows. In the controlled random split, 1,272 out of 2,000 testing observations were identical to complete rows already present during training. In addition, 1,427 testing observations used domains that also appeared during training. Therefore, the original random split does not provide a strong evaluation of performance on genuinely unseen websites.

Duplicate removal also changed the apparent class distribution substantially. The original dataset appears balanced because it contains 5,000 observations from each class. After removing exact duplicated rows, only 529 legitimate observations remained, compared with 3,845 phishing observations. This demonstrates that the original row-level balance depends heavily on repeated legitimate samples.

A stricter domain-aware experiment was therefore performed. Each domain was assigned entirely to either the training set or the testing set. Under this evaluation, no complete row and no domain appeared in both sets.

At the default classification threshold of 0.50, the full-feature XGBoost model achieved an accuracy of 87.5%. However, this value was misleading because the model incorrectly blocked 99 out of 112 legitimate websites. Its specificity was only 11.6%.

A separate domain-aware validation set was then used to select a more balanced threshold of 0.98. At this operating point, the model correctly allowed 104 out of 112 legitimate websites and detected 489 out of 753 phishing websites.

Metric	Default Threshold: 0.50	Validation-Selected Threshold: 0.98
Accuracy	0.875	0.686
Balanced Accuracy	0.552	0.789
Precision	0.883	0.984
Recall	0.988	0.649
Specificity	0.116	0.929
MCC	0.222	0.392
False Positives	99	8
False Negatives	9	264

This comparison illustrates an important cybersecurity trade-off. A lower threshold detects more phishing websites but incorrectly blocks many legitimate websites. A higher threshold reduces disruption for legitimate users but allows more phishing websites to pass undetected. The appropriate operating point depends on whether the system produces a warning, supports analyst triage, or automatically blocks access.

The feature-engineering analysis also identified important limitations. The model relies heavily on a binary URL-length indicator. In the supplied dataset, long URLs are associated more strongly with legitimate observations than with phishing observations. This suggests that the model may partly learn characteristics of the data-collection process rather than universal phishing patterns.

Controlled tests of the standalone feature-extraction script also revealed several implementation issues, including incomplete IP-address detection, unreliable iframe detection, unclear right-click encoding, a mismatch between the declared and returned fields, and silent fallback values when WHOIS functionality is unavailable.

The proposed project has clear strengths. It addresses an important cybersecurity problem, uses interpretable predictors, provides a processed dataset, and offers a

useful educational introduction to phishing detection with machine learning. Its main XGBoost result is approximately reproducible.

However, the project should not be recommended as a deployment-ready phishing detector in its current form. Before applying a similar approach to a real system, the dataset should be deduplicated, complete URLs and timestamps should be preserved, domain-aware and time-aware evaluation should be used, extraction functions should be corrected, and additional independent validation data should be introduced.

The final conclusion is that the selected source is a useful educational baseline, but its reported accuracy alone does not provide sufficient evidence that the proposed methodology will generalize reliably to unseen websites or perform effectively in a real-world environment.

References

- [1] S. G. Sundari, "Phishing Website Detection by Machine Learning Techniques," GitHub repository. Available: <https://github.com/shreyagopal/Phishing-Website-Detection-by-Machine-Learning-Techniques/>
Accessed: June 11, 2026.
- [2] PhishTank, "PhishTank — Join the Fight Against Phishing." Available: <https://www.phishtank.net/>
Accessed: June 11, 2026.
- [3] Canadian Institute for Cybersecurity, University of New Brunswick, "URL Dataset — ISCX-URL2016." Available: <https://www.unb.ca/cic/datasets/url-2016.html>
Accessed: June 11, 2026.
- [4] Scikit-learn Developers, "Metrics and Scoring: Quantifying the Quality of Predictions." Available: https://scikit-learn.org/stable/modules/model_evaluation.html
Accessed: June 11, 2026.
- [5] Scikit-learn Developers, "average_precision_score." Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html
Accessed: June 11, 2026.
- [6] Scikit-learn Developers, "StratifiedGroupKFold." Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedGroupKFold.html
Accessed: June 11, 2026.
- [7] XGBoost Developers, "XGBoost Documentation." Available: <https://xgboost.readthedocs.io/>
Accessed: June 11, 2026.